Data Compaction

Presentation by:
Joe Louis Paul I

Data Compaction

- To remove the redundancy present in the data prior to transmission.
- Use
  - To achieve efficient communication
  - Also known as *lossless compression*.

How to achieve Data Compression?

- To achieve data compression, source coding schemes are applied.
- Different Source Coding Schemes
  - Prefix Coding
  - Huffman Coding
  - Shannon-Fano Coding
  - Lempel-Ziv (LZ) Coding

Prefix Coding

- Prefix
  - Initial part of the codeword.
- Prefix Code Definition
  - Any codeword should not be prefix of some other code words.
- Example

Symbol	Codeword
A	0
B	01

- The codeword of A is the prefix of the codeword of B. This coding scheme is not a prefix code.

Illustrating the Definition of Prefix Coding (1/4)

1. Consider a discrete memoryless source (DMS) with source alphabet $S=\{s_0,s_1,\ldots\ldots\ldots,s_{K-1}\}$, where K is the radix of the source, and the corresponding probability distribution is given by $P=\{p_0,p_1,\ldots\ldots\ldots,p_{K-1}\}$.

Illustrating the Definition of Prefix Coding (2/4)

1. Let the codeword assigned for symbol s_k is $m_{k1}m_{k2}m_{k3}.....m_{kn}$ where $m_{k1}m_{k2}m_{k3}.....m_{kn}$ is the <u>*block of 0's and 1's*</u> , where n is the total number of bits involved in the code.

Illustrating the Definition of Prefix Coding (3/4)

1. Let the prefix of the codeword assigned for symbol s_k is $\underbrace{m_{k1}m_{k2}m_{k3}\ldots m_{kl}}_{\textbf{Prefix}}\ldots\ldots m_{kn}$

Illustrating the Definition of Prefix Coding (4/4)

1. For a prefix of any codeword, the condition is
   - The number of bits in the prefix of the *n* bit codeword is $\leq n$.
   - Hint:
     - *<u>Entire codeword can also be taken as a prefix.</u>*

Example: Prefix Coding

Source Symbol	Probability	C_1	C_2	C_3
S0	0.5	0	0	0
S1	0.25	01	10	1
S2	0.125	011	110	00
S3	0.125	0111	111	11

Note: C_1 and C_3 are not prefix codes. Any codeword should not be a prefix of another codeword. Hence, ***C_2 is a prefix coding scheme***.

Characteristics of Prefix Coding

- Prefix codes are ***uniquely decodable*** codes.
- Prefix codes are also ***instantaneous cods.***

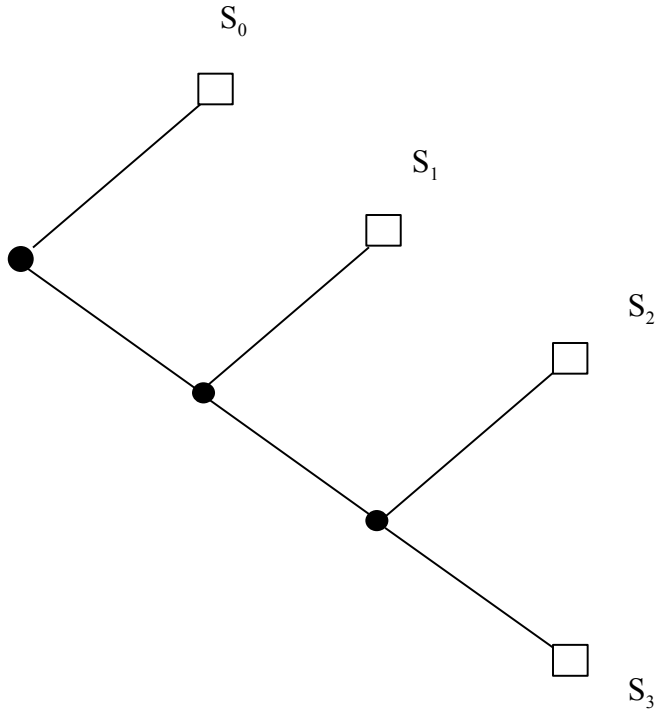
Decision Trees (1/2)

- A graphical representation used by the source decoder to decode the sequence of codewords generated by the prefix source code.
- That is, decision trees are used at the destination side.

Decision Trees (2/2)

- A decision tree consists of,
  i. One initial state (or) first decision point (●).
  iii. Terminal states (□ or ○)
     - Depends on number of source symbols

Constructing Decision Trees for a Prefix Code (for C_2)



Decision Tree Algorithm (1/4)

- Consider the encoded binary sequence obtained at the receiver is

  1011111000……
- Decision tree algorithm used by the source decoder.
- *Input:* The encoded binary sequence (sequence of codewords)
- *Output:* Reconstructed original message.

Decision Tree Algorithm (2/4)

Step 1: The source decoder always starts at initial state of the decision tree.

Step 2: The first received bit moves the decoder to the terminal state $\mathbf{S_o}$ if it is **0** or else to **the second decision point** if its is **1**.

Decision Tree Algorithm (3/4)

Step 3 : Now, the second received bit moves the decoder one step further down in the tree either to the terminal state S_1, if it is 0 or to the third decision point if it is 1.

Step 4: This process will be continued further for the remaining code works.

Decision Tree Algorithm (4/4)

Step 5 : Once each terminal state emits its symbol, the decoder is reset to the initial .

Step 6: Every bit is examined only Once in the tree [It is the specialty of the source decoder] .

Prefix Codes Vs Reversible Codes (1/2)

- *A prefix code is always uniquely decodable. But the converse is not necessarily true.*
- For example, code **C_1 does not satisfy the prefix condition**, yet it is uniquely decodable since the bit 0 indicates the beginning of each codeword in the code.

Prefix Codes Vs Reversible Codes (2/2)

- Exercises:
- Consider a encoded binary sequence, 01110. Try to find out the source or original message using Code III.
- Find the source message for 0111010001 using Code I.

- To decode the message, you'll need to know the specific encoding scheme of **Code III**. It could involve using a predefined mapping or dictionary for this code that converts binary sequences to letters or symbols.
- If "Code III" is similar to a Huffman code or any other variable-length prefix code, you would sequentially match the binary string `01110` with the appropriate prefix or group of bits.
- If this code uses a dictionary, you should look up `01110` in the codebook and retrieve the corresponding symbol.

2. **Second Exercise:**
   - The encoded binary sequence is `0111010001`.
   - You are asked to find the source or original message using **Code I**.

   Approach:
   - Similarly, to decode `0111010001`, you would use the encoding scheme for **Code I**.
   - If **Code I** involves fixed-length codes or a similar method, you would split the sequence into segments and decode each segment.
   - Again, you would need to consult a codebook or lookup table if **Code I** follows a specific coding scheme.

Key Steps in Decoding:

- **Identify the Coding Scheme**: Knowing the structure of Code I and Code III is essential. Is it a fixed-length code, Huffman code, or something else?
- **Match the Binary to Symbols**: For each binary sequence, you match the binary codes to the corresponding symbols or characters in the given codebook.

Once you have the codebooks or encoding schemes, you can apply these steps to decode the binary sequences. If you have those details (like code tables), feel free to share them, and I can assist you further.